



Compilers

Guillermo A. Pérez

Lecture 4: Top-down parsing and parser generators

TL;DR: This lecture in short

How do we go from grammars to a deterministic parser?

We know a CFG can be translated into a nondeterministic PDA. We will see how to use **lookahead** to obtain a deterministic top-down parsing algorithm.

Main references

- **Introduction to Language Theory and Compilers.** Gilles Geeraerts and G. A. Pérez. 2019.
- **Crafting a Compiler.** Charles N. Fischer et al. 2010. Pearson Education.
- **Compiler Design.** Helmut Seidl et al. 2012. Springer
- **Compilers: Principles, Techniques, Tools.** Alfred Aho et al. 2006. Pearson Education.
- Online: Wikipedia, LLVM, ANTLR, ...

Required and target competences

What tools do we need?

Formal language theory (languages and machines, machines and computability);

What skills will we obtain?

- theory: look-ahead pushdown automata
- practice: how to implement LPDA-based **predictive** parsers

How will these skills be useful?

- It will allow you to build your compiler without hand-crafting a parser.

Outline

1. Top-down parsers: from CFGs to PDAs
2. From PDAs to look-ahead PDAs
3. First and Follow
4. (Strong) LL(k) grammars
5. LL(k) languages
6. LL(k) parsers

Top-down parsers

Parsing from the top

Such a parser tries to build a **derivation tree** starting from the root. It “grows” the tree by applying grammar rules one after the other so that, in the end, the sequence of tree leaves forms the desired string.

Intuition

Remember: Every CFG G can be transformed into an equivalent PDA A that simulates the derivation process in a left-most manner from S .

- How do we obtain a deterministic top-down parsing algorithm from that construction?

Top-down parsers: background

Some background

- Historically, compilers such as gcc were written by “hacking” the code of an automatically generated bottom-up parser.
- Recent versions of gcc and clang use hand-written top-down parsers!
- ANTLR: ANother Tool for Language Recognition generates top-down parsers.

From a CFG to a PDA

Reminder

- Remember the grammar and PDA for arithmetic expressions (Figs 4.4, 4.14 lecture notes)

(1)	Exp	→	Exp + Exp
(2)		→	Exp * Exp
(3)		→	(Exp)
(4)		→	Id
(5)		→	Cst

- Can you make the grammar unambiguous?

From a CFG to a top-down parser

Algorithm

The idea is to keep in the stack the current sentential form: left = top.

- **Produce:** For all rules $A \rightarrow \alpha$, the PDA has a transition that pops A and pushes α into the stack (without reading from the input).
- **Match:** For all terminals $a \in T$, there is a transition that pops a and reads a from the input.

The PDA accepts by **empty stack**.

The PDA is NOT implementable

Unfortunately, we cannot implement an efficient simulation of the PDA since it is **nondeterministic**.

- If there are rules $A \rightarrow \alpha_1, A \rightarrow \alpha_2$, which one should we produce?

Predictive parsers

Look-ahead

Remember: **Look-ahead** corresponds to having access to more than one letter of the input at a time **without** consuming it.

- Parsers that make use of look-ahead are said to be **predictive**.

How much look-ahead?

The amount k of characters we **need** to see before deterministically choosing a production rule depends on the grammar.

- A single character of look-ahead suffices for the following.

(1)	S	$\rightarrow$	a
(2)		$\rightarrow$	b

Look-ahead pushdown automata

Definition (k -look-ahead PDA)

A k -LPDA A is a tuple $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ where everything is as for PDAs except for $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \times \Sigma^{\leq k} \rightarrow 2^{(Q \times \Gamma^*)}$ where

$$\Sigma^{\leq k} = \bigcup_{i=0}^k \Sigma^i$$

is the set of all words of length at most k on the alphabet Σ .

LPDA configurations

Configuration change

The LPDA A can move from $(q, auv, X\beta)$ to $(q', uv, \alpha\beta)$ if there is $(q', \alpha) \in \delta(q, a, X, au)$ and

- $X \in \Gamma$,
- $a \in \Sigma \cup \{\varepsilon\}$,
- $u \in \Sigma^{\leq k-1}$,
- $v \in \Sigma^*$, and
- if $|auv| \geq k$ then $|au| = k$, otherwise $v = \varepsilon$.

Exercise: LPDAs

Exercise

- Draw the 1-LPDA that recognizes the previous simple grammar.
- Give the formal description of the LPDA according to the previous definition.

Expressiveness of LPDAs

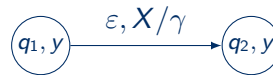
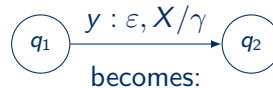
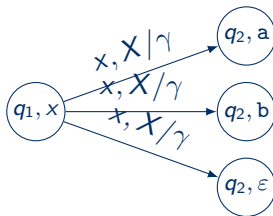
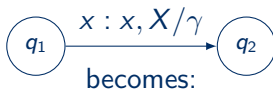
Theorem (LPDAs recognize all CFLs)

*Since LPDAs extend PDAs, and for every CFL there is a PDA that recognizes it, we have that **for every CFL there is an LPDA that recognizes it.***

Theorem (Any language recognized by an LPDA is a CFL)

*Since the look-ahead is a bounded amount of information, we can **guess it, store it, and check it** with a PDA. Hence, every LPDA can be transformed into a PDA and thus recognizes a CFL.*

From LPDAs to PDAs (graphically)



Exercise

From LPDA to PDA

- The idea works for $k = 1$: $L(G) = L(G')$
- Construct the PDA for the simple grammar from earlier.

First and Follow

Remind the lecturer: COPY ON THE BOARD!

Which produce?

Based on 1 character of look-ahead, how can we decide which rule to produce?

(1)	A	$\rightarrow$	aaa
(2)		$\rightarrow$	Bbb
(3)		$\rightarrow$	Cdd
(4)	B	$\rightarrow$	b
(5)	C	$\rightarrow$	c
(6)		$\rightarrow$	ε

Building a look-ahead table (1/2)

From A

We have to choose from rules 1–3. Note that

- All words generated from A , using rule 1 as first rule, will start with a .
- All words generated from B will start by b , so all words generated from A using rule 2 as first rule, will start with b .
- C can generate c and ε . So words generated using rule 3 will start with c or dd .

Look-ahead				
Var	a	b	c	d
A	1	2	3	3

Building a look-ahead table (2/2)

From B

We have no choice.

From C

- $\{c, \varepsilon\}$ is the set of all words generated from C .
- When do we apply the rule 6: $C \rightarrow \varepsilon$? It seems we need to consider the **context** of C .
 - In all words generated from a sentential form containing C , the word generated from C will be followed by d .

Var	Look-ahead			
	a	b	c	d
A	1	2	3	3
B		4		
C			5	6

Formal definitions

Definition (First)

For α a sentential form, we define

$$\text{First}^k(\alpha) := \{w \in T^* : \alpha \Rightarrow^* wx \wedge (|w| = k \text{ or } |w| < k \wedge x = \varepsilon)\}.$$

Definition (Follow)

For α a sentential form, we define

$$\text{Follow}^k(\alpha) := \{w \in T^* \mid \exists \beta, \gamma : S \Rightarrow^* \beta\alpha\gamma \wedge w \in \text{First}^k(\gamma)\}.$$

Computing First

First

To compute $\text{First}^k(\alpha)$ for some sentential form $\alpha = X_1X_2 \dots X_n$ we can

1. Compute $\text{First}^k(X_1)$. If all words are of length k then we are done.
2. Otherwise (in particular if $\varepsilon \in \text{First}^k(X_1)$), we need to complete those elements of $\text{First}^k(X_1)$ that are not 'long enough' by elements from $\text{First}^k(X_2)$.
3. Again, this might not be sufficient, so we compute $\text{First}^k(X_3)$, etc.

Computing Follow

Follow

Every time we have a rule of the form $A \rightarrow \alpha B \beta$ we do the following.

1. We can add more information to $\text{Follow}^k(B)$: a string generated by B can be followed by a string generated by β , so we can use $\text{First}^k(\beta)$
2. Observe the words in $\text{First}^k(\beta)$ might be shorter than k , so we might need to complete them with $\text{Follow}^k(A)$.

Break

Questions? Coffee? Tea?

Exercises: first and follow

(1)	S	$\rightarrow$	$\text{Exp}\$$
(2)	Exp	$\rightarrow$	$\text{Prod Exp}'$
(3)	Exp'	$\rightarrow$	$+\text{Prod Exp}'$
(4)		$\rightarrow$	$-\text{Prod Exp}'$
(5)		$\rightarrow$	ε
(6)	Prod	$\rightarrow$	$\text{Atom Prod}'$
(7)	Prod'	$\rightarrow$	$*\text{Atom Prod}'$
(8)		$\rightarrow$	$/\text{Atom Prod}'$
(9)		$\rightarrow$	ε
(10)	Atom	$\rightarrow$	$-\text{Atom}$
(11)		$\rightarrow$	Cst
(12)		$\rightarrow$	Id
(13)		$\rightarrow$	(Exp)

Compute $\text{First}^2(\text{Atom})$ and $\text{Follow}^1(\text{Prod})$

Why LL(k) grammars?

First and Follow for deterministic parsers

Some PDAs can be made deterministic using a k -look-ahead. Using First and Follow, we will define a **semantic restriction** of CFLs which is guaranteed to yield such a deterministic k -LPDA.

The deterministic machine

The deterministic LPDA reads the input left-to-right and uses left-most derivations. It is therefore **Left scanning** and **Left parsing**.

LL(k) grammars

Definition (LL(k) grammar)

A CFG is LL(k) iff for all pairs of derivations

$$S \Rightarrow^* wA\gamma \Rightarrow w\alpha_1\gamma \Rightarrow^* wx_1$$

$$S \Rightarrow^* wA\gamma \Rightarrow w\alpha_2\gamma \Rightarrow^* wx_2$$

with $w, x_1, x_2 \in T^*$, $A \in V$, and $\gamma \in (V \cup T)^*$, if $\text{First}^k(x_1) = \text{First}^k(x_2)$ then $\alpha_1 = \alpha_2$.

- **Oversimplifying:** if the constructed k -LPDA is deterministic, the grammar is LL(k)

How do we know?

Two things to avoid to be LL(k)

- First/First conflict: two grammar rules for the same variable have intersecting First sets (of the right-hand sides).
- First/Follow conflict: the First and Follow sets of the right-hand side of two grammar rules for the same variable intersect.

Solutions

Left factoring (for First/First conflicts); Substitution (for First/Follow conflicts, but may introduce First/First conflicts); Left recursion removal

Example: LL(k) grammars

(1)	S	$\rightarrow$	aAa
(2)	S	$\rightarrow$	$bABa$
(3)	A	$\rightarrow$	b
(4)	A	$\rightarrow$	ε
(5)	B	$\rightarrow$	b
(6)	B	$\rightarrow$	c

- Is the above grammar $LL(1)$? Why not?
- Is it $LL(2)$?

Strong LL(k) grammars

Definition (Strong LL(k) grammars)

A CFG is **strongly** LL(k) iff for all pairs of rules $A \rightarrow \alpha_1$ and $A \rightarrow \alpha_2$ with $\alpha_1 \neq \alpha_2$:

$$\text{First}^k \left(\alpha_1 \cdot \text{Follow}^k(A) \right) \cap \text{First}^k \left(\alpha_2 \cdot \text{Follow}^k(A) \right) = \emptyset.$$

Theorem (Strength)

Every strong LL(k) grammar is LL(k). Every LL(1) grammar is strongly LL(1).

Exercise

- Argue that the grammar from the previous slide is **not** strongly LL(2).

LL(k) languages

Definition (LL(k) language)

A language L is LL(k) if there exists an LL(k) grammar G such that $L(G) = L$.

(1)	S	$\rightarrow$	aba
(2)		$\rightarrow$	aa
(3)		$\rightarrow$	bbba
(4)		$\rightarrow$	bbca
(5)		$\rightarrow$	bba
(6)		$\rightarrow$	bca

Exercises

- Is it equivalent to the **non**-LL(1) grammar from before?
- Is its language LL(1)?

A hierarchy of $LL(*)$ languages

Theorem (A strict hierarchy)

$$\dots \subset LL(k) \text{ langs.} \subset LL(k+1) \text{ langs.} \subset \dots$$

LL(1) parser

From a grammar to a parser

We now know about LL(k) grammars and **some tricks** to try to get an LL(k) grammar for a language. How do we **algorithmically** construct a parser from it? We build an **action table**.

Which tricks?

- Ambiguity: if a grammar is ambiguous, then it cannot be LL(k). Setting priorities for arithmetic operations helped us with this problem before.
- Left recursion: we can turn left recursion into right recursion.
- Factor out common prefixes (to avoid First/First conflicts).

The action table for LL(1) parsers

Using First and Follow

The **action table** T maps a top-of-stack symbol and a look-ahead to **actions**.

- Accept, Reject, Match
- Produce rules

$T(A, a)$ contains the rule $A \rightarrow \beta$ if and only if

- a is in $\text{First}^1(\beta)$ or
- ε is in $\text{First}^1(\beta)$ and a is in $\text{Follow}^1(A)$

More generally: iff $a \in \text{First}^1(\beta \cdot \text{Follow}^1(A))$.

What do we want in a table?

Is the table good for parsing?

If the table contains at most one rule for every pair (A, a) then the parser knows what to do (either a **unique produce** or **match** if the top of the stack is a terminal) at every step. So the grammar was LL(1).

LL(1) parsing example

Go back to see the referenced grammar: Fig 5.4 in lecture notes

Important note

We will assume the stack starts with SZ_0 ; and that a terminal on the top of the stack asks for a Match (except for the EOF symbol \$, which asks for Match and accept).

T	\$	+	-	*	/	Cst	Id	(	)
S			1			1	1	1	
Exp			2			2	2	2	
Exp'	5	3	4						5
Prod			6			6	6	6	
Prod'	9	9	9	7	8				9
Atom			10			11	12	13	

Incorrect parse

Error reporting

- What are the contents of the “stack” when you get stuck trying to parse `Id(Id)$`?
- What look-ahead symbol would allow you to continue?

What about LL(k) action tables?

Complicated: easier to first strengthen your grammar

Until the mid-1990s, it was widely believed that $LL(k)$ parsing (for $k > 1$) was impractical, since the parser table would have exponential size in k in the worst case. This perception changed gradually after the release of the Purdue Compiler Construction Tool Set around 1992, when it was demonstrated that many programming languages can be parsed efficiently by an $LL(k)$ parser without triggering the worst-case behavior of the parser.

ANTLR

The Purdue Compiler Construction Tool Set (PCCTS) is the predecessor of ANTLR!

Recursive-descent parsers

Simple rules

Instead of pushing sentential forms into a stack, we can make **recursive calls** to functions handling nonterminals. Consider, e.g., a grammar with rules $S \rightarrow Ab$ and $S \rightarrow Bc \dots$

```
1 def S():
2     # This is the look-ahead
3     n = get_next_character()
4     # If n is in First(A b Follow(S))
5     if n == 'a1' or ... or n == 'an' :
6         r = A()
7         if (!r): return False
8         n = read_next_character() # Consume
9         if (n == 'b'): return True
10        else: return False
```